

CS 340: Programming Language Design

Dian Chen

Fall 2026

Introduction

- About me
 - Ph.D. in Computer Science, *Virginia Tech*
 - Research: AI-based Smart Systems, Machine Learning, Explainable AI
- **About you**
 - Your name
 - Your major
 - Favorite programming language (or the one they've used the most) and why

About This Course

- Why Study Programming Languages?
 - Different ways of expressing computation
 - Why some languages are better for certain problems
 - How compilers/interpreters understand code
 - How language features affect correctness and productivity

Goal: Become a better programmer by understanding language design.

Syntax

- The **syntax** of a programming language is the set of rules that describe which sequences of characters form legal programs.
- Programming language syntax has **two components**
 - **Lexical structure**
 - How characters are grouped into words (lexemes)
 - **Syntactic structure**
 - How lexemes are organized into valid statements and programs

Key Idea: A program must satisfy **both** lexical and syntactic rules.

Lexical vs. Syntactic Structure

Lexical Structure	Syntactic Structure
Characters → Tokens	Tokens → Statements
Scanner (Lexer)	Parser
Words	Sentences

Example:

```
int count = total + 1;
```

Lexical analysis produces:

```
[int] [count] [=] [total] [+] [1] [;]
```

Parser checks whether these tokens form a valid statement.

Syntax vs. Semantics

- A program may be
 - Lexically correct
 - Syntactically correct

but still be **semantically incorrect**.

- **Example: `int count = "Hello, world";`**
 - Lexically valid? Yes
 - Syntactically valid? Yes
 - Semantically valid? Semantic error: assigning a string to an integer.

Tokens

- **Lexeme:** The actual sequence of characters in the source code.
- **An *identifier*:** a name which can refer to a construct such as a variable, procedure, data type, etc.
 - foo, bar, sheep, LinkedList
- A **token** is a **category of lexemes**.
 - foo, bar, sheep, and LinkedList are different lexemes, but they all belong to the **Identifier** token.

The *grammar* of a programming language is described by specifying **what tokens may legally appear** in each syntactic construct.

Formal Languages, Recognizers & Generators

- A **formal language** is a set of strings of symbols.
 - Alphabet: {a,b}
 - String: a, ab, bba, ϵ (empty string)
 - Language: {a, aa, aba}
- Two ways to describe a formal language:
 - **Recognizer** determines: Is this string in the language?
 - Input: abba $\rightarrow$ output: Yes/No
 - **Generator** produces: All valid strings in the language
 - a^* generates ϵ , a, aa, aaa, aaaa, ...

Regular Languages

- *Regular* languages are a useful model for the lexical structure of a programming language.
 - Regular expression: a *generator* for a regular language
 - Finite automaton: a *recognizer* for a regular language

Regular Expressions

- A regular expression generates a regular language.
- Basic regular expressions
 - Any symbol in the alphabet
 - Empty string (ϵ)
- Rules for combining regular expressions
 - Concatenation
 - Alternation
 - $a \mid b \rightarrow a \text{ or } b$
 - Repetition
 - Kleene Star (*): $a^* \rightarrow \{\epsilon, a, aa, aaa, \dots\}$
 - Kleene Plus (+): $a^+ \rightarrow \{a, aa, aaa, \dots\}$

Example

Regular expression	Language (set of strings)
a	{ a }
aa	{ aa }
a*	{ ϵ , a, aa, aaa, ... }
aa*	{ a, aa, aaa, ... }
a+	{ a, aa, aaa, ... }
ba+	{ ba, baa, baaa, ... }
(ba)+	{ ba, baba, bababa, ... }
(ab)	{ a, b }
alb*	{ a, ϵ , b, bb, bbb, ... }
(ab)*	{ ϵ , a, b, aa, ab, ba, bb, ... }
aa(ba)*bb	{ aabb, aababb, aabababb, ... }

Regular Expressions in Programming Languages

- Regular expressions describe token patterns.
- The regular expression generates all legal decimal integer literals in C and C++:

$(-|\epsilon)(0|1|2|3|4|5|6|7|8|9)^+$

- Meaning
 - Optional minus sign
 - One or more digits

Strategy

- **Break the language into smaller pieces.**
- **Example:** Strings with alternating a's and b's
 1. Break into four subsets
 - Even length starting with a
 - Odd length starting with a
 - Even length starting with b
 - Odd length starting with b
 2. Regular expressions
 - $(ab)^*$
 - $(ab)^*a$
 - $(ba)^*$
 - $(ba)^*b$
 3. Combined: $(ab)^* \mid (ab)^*a \mid (ba)^* \mid (ba)^*b$

Practice

- All strings that begin with a , over $\{a, b\}$
 - Regular expression: $a(b \mid a)^*$
- All strings contain exactly one a , over $\{a, b\}$
 - Regular expression: b^*ab^*
- All strings contain at least one a , over $\{a, b\}$
 - Regular expression: $(a \mid b)^* a (a \mid b)^*$
- All strings that begin and end with the same character over $\{a, b\}$
 - Regular expression: $a(a \mid b)^* a \mid b(a \mid b)^* b \mid a \mid b$

Key Takeaways

- Syntax vs. semantics
- Lexical structure vs. syntactic structure
- Lexemes and tokens
- Formal languages
- Regular expressions
- Regular languages